MODULE 3 · MACHINE LEARNING · TOPICS 3.1 + 3.2

What Machine Learning Is

… and how to feed it words

Day 31 — the first session of Module 3. Nothing is assumed. Every term is defined when it first appears.

1	3	5
A new way of programming	Kinds of learning, drawn out	Ways to turn words into numbers



Boston University

Prepared & presented by **Srinivasa Sai Chava**

WELCOME TO MACHINE LEARNING

Module 3 starts here, and the job changes

Modules 1 and 2 — 30 days

- You wrote the rules yourself, in Python
- You described data that already existed
- You tested claims about a population
- Every answer came from a formula you could write out by hand

Module 3 — from today

- The computer finds the rule, from examples
- You judge it on data it has never seen
- You will not be able to write the rule by hand — and that is the point
- Your job becomes choosing, feeding and checking the model

Three promises for today

1. You will see a model learn a rule in front of you, with no magic in it
2. A model will score 100% on its homework and FAIL the exam — worse than guessing
3. Writing one word as a number the wrong way will double a model's error

SESSION PLAN

How today's two hours are structured

1. **What machine learning is** — 20 min
   The paradigm, and the vocabulary
2. **The three kinds of learning** — 25 min
   Supervised, unsupervised, reinforcement
3. **How a project actually runs** — 15 min
   The pipeline, and train/test
4. **What a feature is** — 10 min
   And why models cannot read words
5. **Encoding categorical data** — 20 min
   Label, one-hot, ordinal, and the traps
6. **Practice notebook** — 30 min
   You encode, you split, you check

Stop me the moment a word goes past that you have not heard before. On the first day of a new module that is not an interruption — it is the lesson.

1

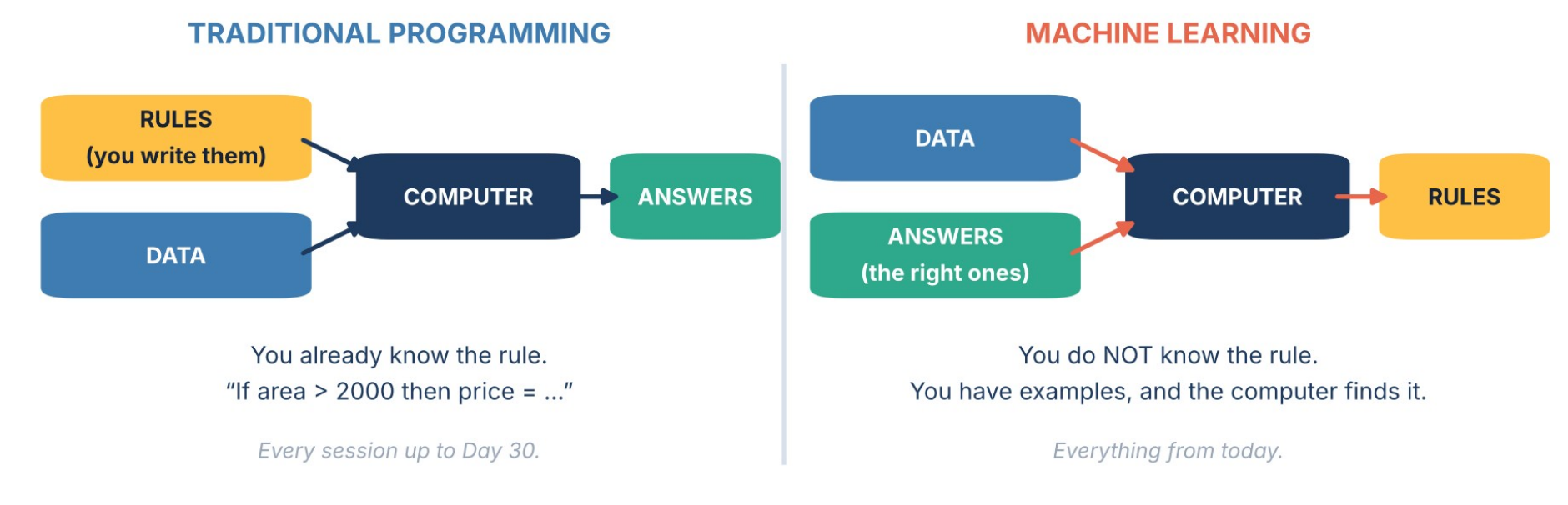
What Machine Learning Is

A different way of writing programs · 20 minutes

- The paradigm flip
- A first example
- The vocabulary
- What “learning” means

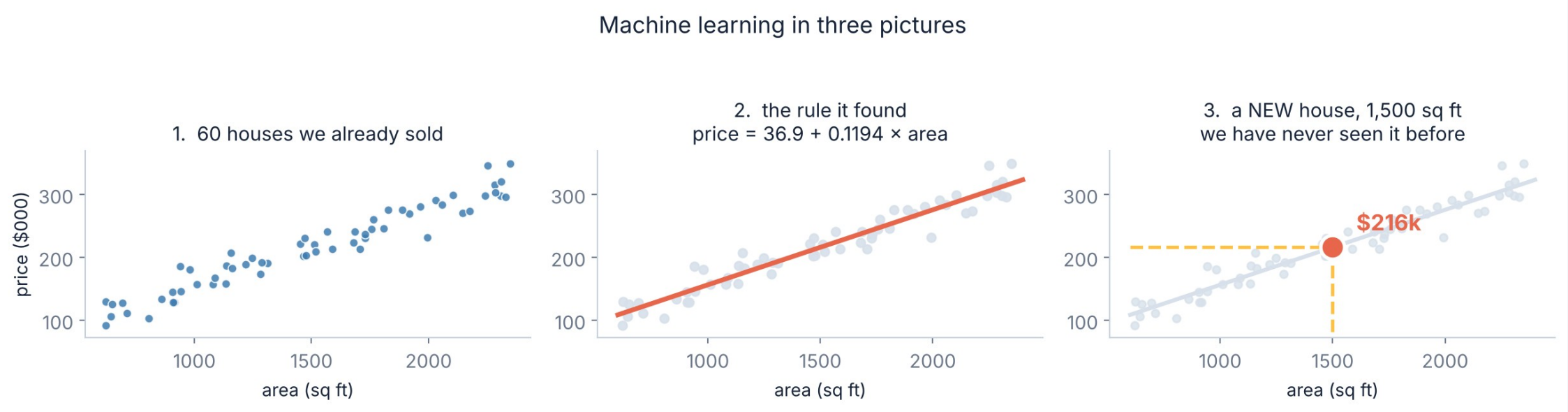
Slides by Srinivasa Sai Chava

⭐ The whole idea, in one picture



Read the two diagrams and notice that RULES and ANSWERS have swapped places. That swap is the entire definition of machine learning. You hand the computer examples of the right answer, and it hands you back a rule — one you could not have written yourself, because you do not know it.

A real one: what is this house worth?



1. The examples

60 houses we already sold. For each one we know the area AND the price — the question and the answer together.

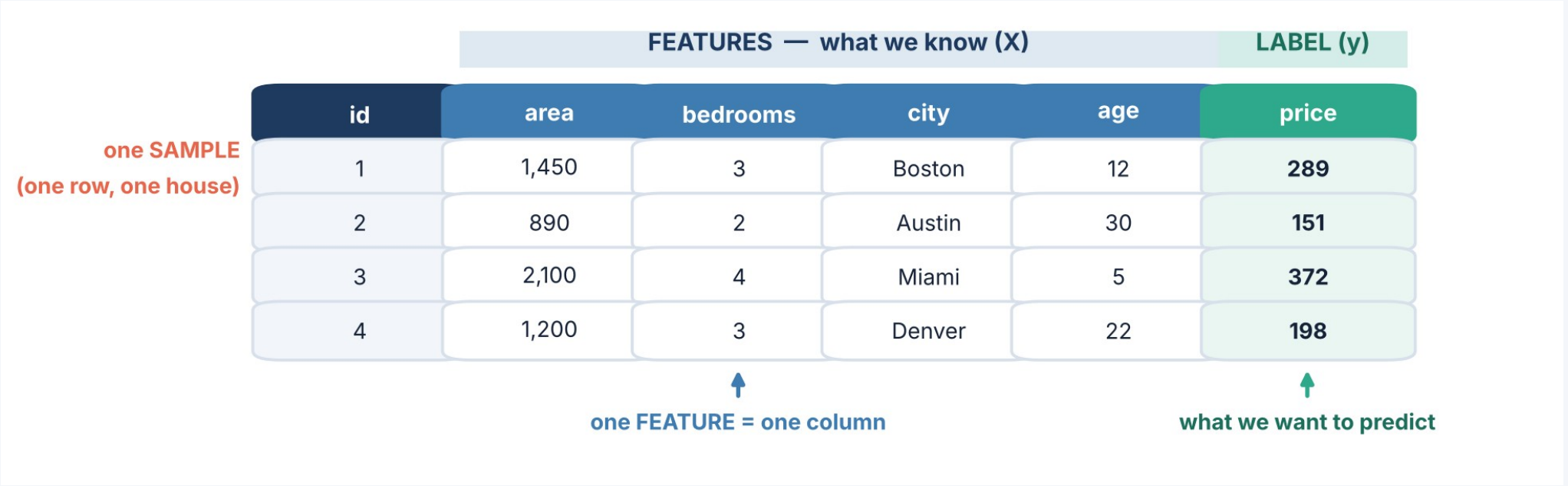
2. The rule it found

Nobody wrote this line. The computer tried millions of lines and kept the one that sat closest to the dots.

3. The point of it all

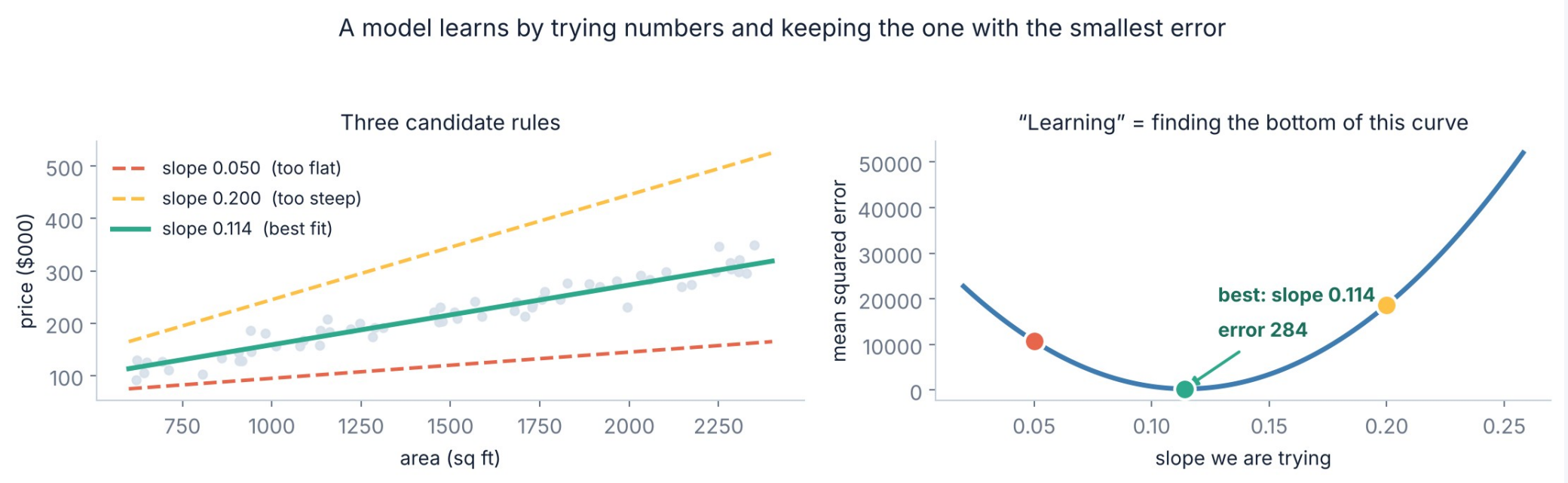
A house we have NEVER seen. The rule gives an answer anyway. That is the only thing we ever wanted.

The five words you need before anything else



`sample`	one row — one house, one customer, one email	`feature`	one input column (X)	`label`	the answer column (y)
`model`	the rule, once it is found	`training`	the act of finding it		

⭐ What “learning” actually means — there is no magic in it



The entire algorithm, in words

Pick numbers. Measure how wrong you are. Change the numbers so you are less wrong. Repeat. That is it — for linear regression and, with more arithmetic, for a neural network too.

Here we tried every slope by brute force

That works with one number to find. With a million, you need a smarter way down the curve — that is GRADIENT DESCENT, and we derive it step by step on Day 35.

2

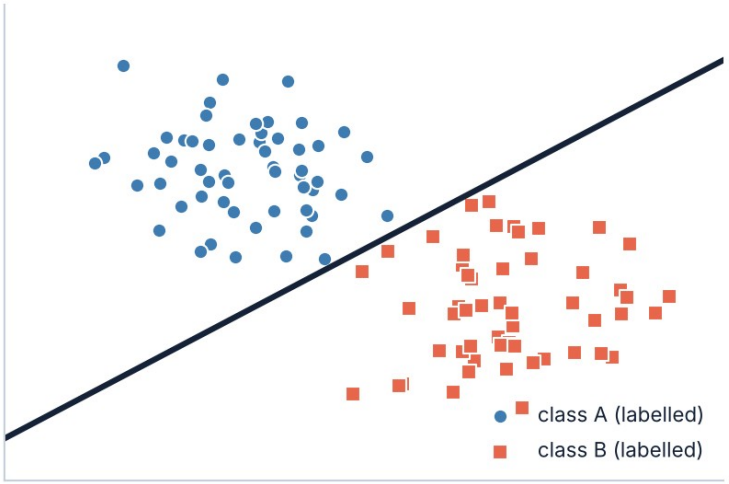
The Three Kinds of Learning

What tells them apart · 25 minutes

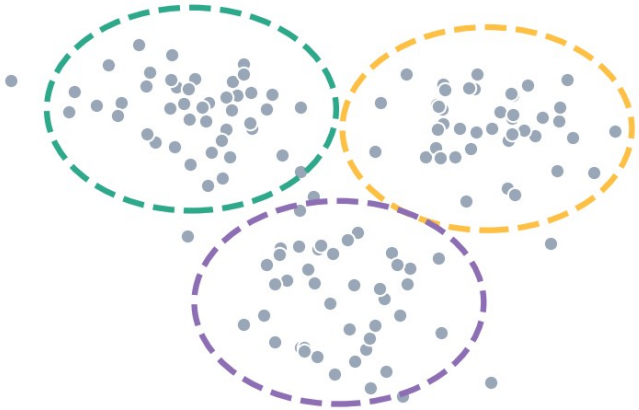
- Supervised
- Unsupervised
- Reinforcement
- Which one is my problem?

⭐ Three kinds of learning, side by side

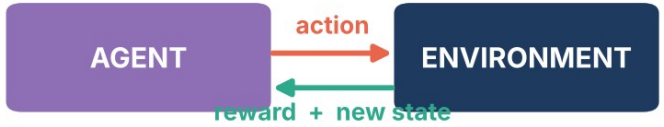
SUPERVISED
the answers come with the data



UNSUPERVISED
no answers — find the structure



REINFORCEMENT
learn by trial and reward



no answer key at all.

Try something, see what happens,

do more of what paid off.

SUPERVISED

Every example comes with its correct answer attached. The model learns to copy the answer key, then applies it to new rows.

~90% of what industry does.

UNSUPERVISED

No answers at all. The model looks for structure that was already in the data — groups, or a simpler description.

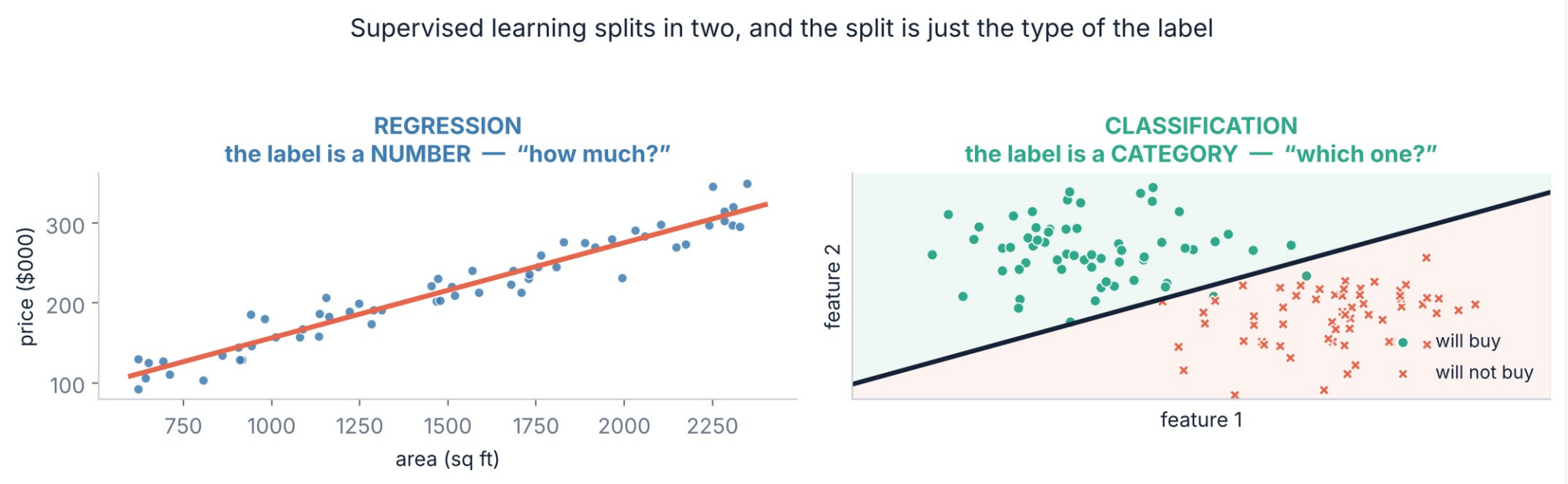
There is no “correct” to score against.

REINFORCEMENT

No answers either, but there IS feedback: do something, get a reward or a penalty, adjust.

Games, robots, recommendation.

Supervised learning splits in two — and the split is just the label



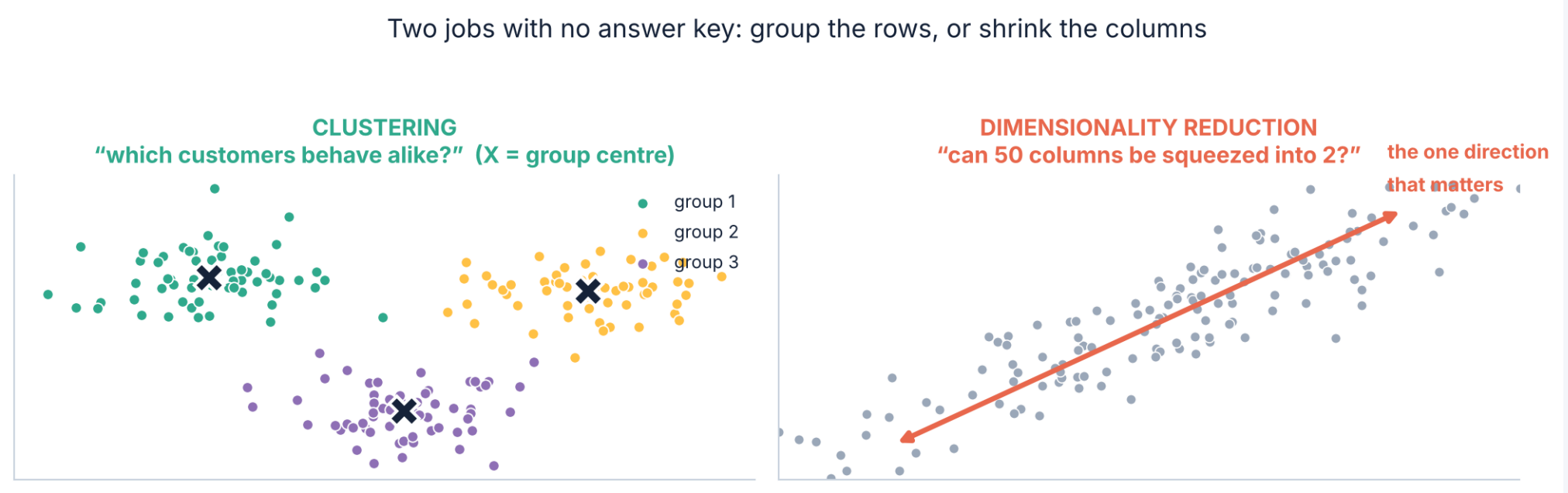
REGRESSION — the label is a number

House price · tomorrow's temperature · how many units we will sell · a patient's blood pressure. The answer can be any value on a scale.

CLASSIFICATION — the label is a category

Spam or not · which of 10 digits · will this customer leave · is this tumour malignant. The answer is one of a fixed list.

Unsupervised: two jobs, no answer key



CLUSTERING — group the rows

“Which of our customers behave alike?” Nobody told the model there were three groups, or what to call them. You name them afterwards, by looking.

DIMENSIONALITY REDUCTION — shrink the columns

“These 50 columns mostly say the same thing — can 2 columns carry it?” Used to plot high-dimensional data, and to speed models up.

Reinforcement learning: nobody says what is right, only what paid off

Learning to ride a bicycle

Nobody gave you a table of handlebar angles. You pushed off, you fell over, and falling over was the penalty. You tried again, you stayed up for three seconds, and that was the reward. Nothing supervised the ride — but the world kept score.

The four pieces

`agent`	the learner — the thing making choices
`environment`	the world it acts in
`action`	what it chooses to do
`reward`	the number the world gives back

Where you have met it without knowing

Game-playing engines that beat world champions at Go and chess · a robot arm learning to grasp · a warehouse robot routing itself · the feed that decides what to show you next and watches whether you keep scrolling · data-centre cooling systems that learn an energy-saving policy.

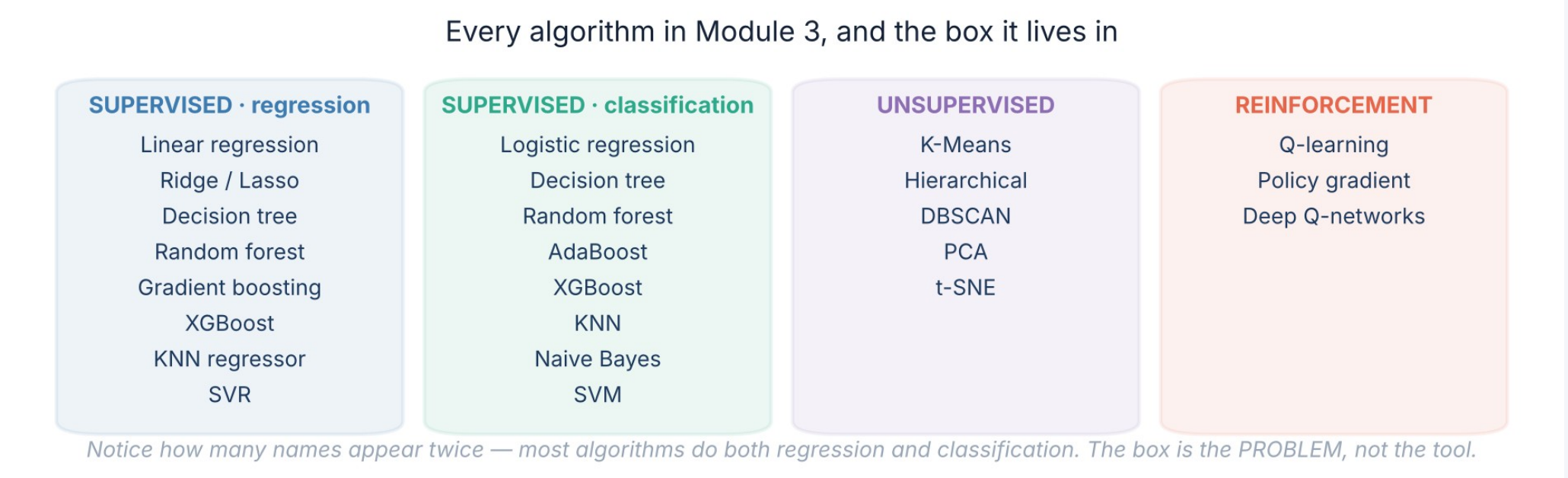
Which kind is my problem? One question decides it.

Look at your data and ask: “Do I have the right answers for the rows I already have?”

What you have	What you want	It is...	Example
Rows WITH the right answer, a number	predict that number	supervised · regression	past sales → next month's sales
Rows WITH the right answer, a category	predict that category	supervised · classification	past emails → spam or not
Rows with NO answers	find natural groups	unsupervised · clustering	customers → segments
Rows with NO answers, too many columns	a simpler description	unsupervised · dim. reduction	80 survey items → 3 scores
No fixed data — a world you can act in	a good policy over time	reinforcement	a robot learning to walk

⚠️ **The most common beginner mistake: having no labels, wanting a prediction anyway, and reaching for a supervised model. If nobody has written down the right answer for your existing rows, no supervised algorithm can help you — getting labels IS the project.**

Every algorithm in this module, and the box it lives in



Do not memorise this — it is a map to come back to. Two things worth noticing now: most of the famous names do BOTH regression and classification, so the box is set by your problem and not by the tool; and the first six weeks of this module live almost entirely in the first two columns.

3

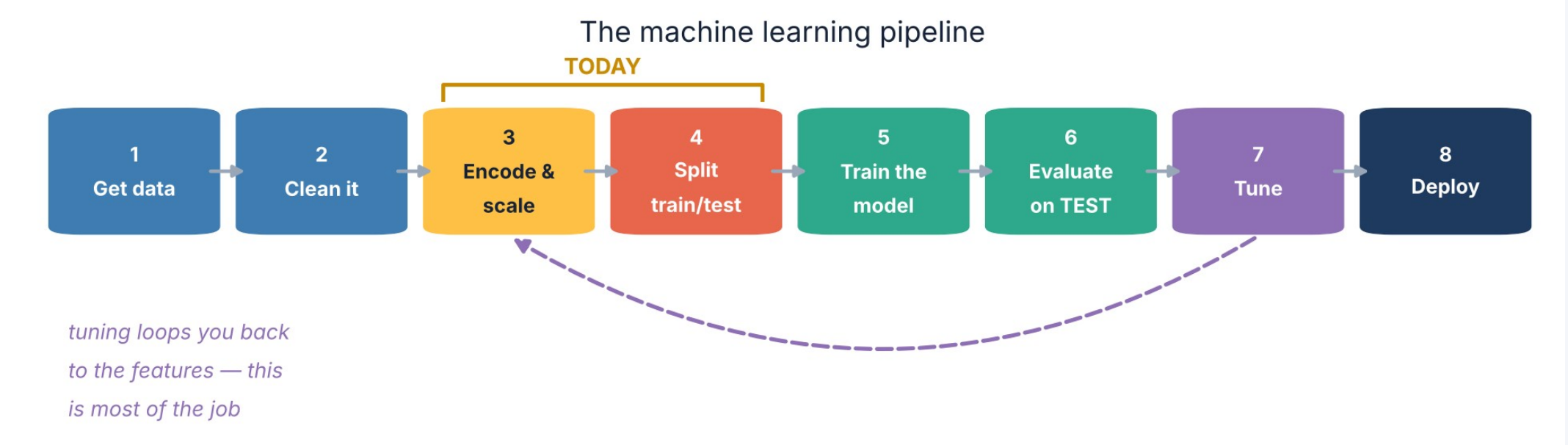
How a Project Actually Runs

The pipeline, and the one rule you cannot break · 15 minutes

- The eight steps
- Train and test

Slides by Srinivasa Sai Chava

The eight steps, every time



Where the time actually goes

Steps 1 to 3 — getting, cleaning and encoding the data — are routinely most of a real project. Step 5, the part everyone imagines when they say “machine learning”, is often three lines of code.

Today we do steps 3 and 4

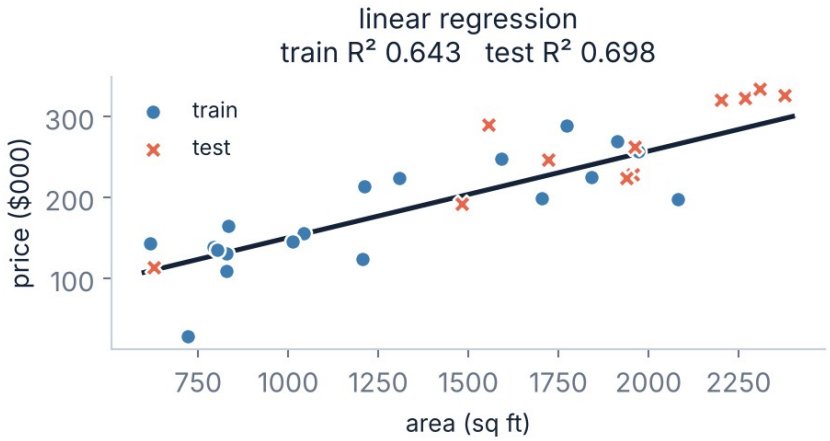
Encoding, and the train/test split. Steps 1 and 2 you already know — that was Module 1. Step 5 onwards is the rest of Module 3.

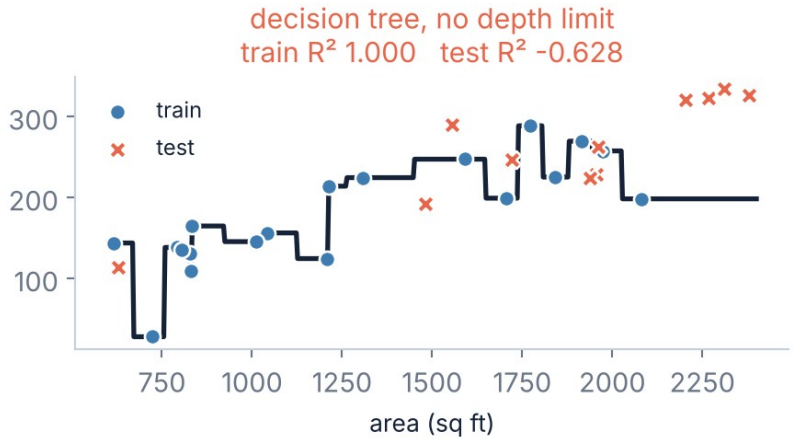
⚠️ The one rule you can never break

The tree scored 100% on data it had already seen, and did WORSE THAN GUESSING THE AVERAGE on new data

split the rows FIRST







What happened on the right

The decision tree drew a line through EVERY training point exactly — $R^2 = 1.000$, a perfect score. On the held-out test points it scored −0.628: worse than if it had ignored the data and guessed the average price every time.

Why we split before we start

Memorising the answers is not learning, and from inside the training data the two look identical. Hiding 20–30% of the rows before you start is the only way to tell them apart.

This is called OVERFITTING, and it is the central problem of the whole module.

4

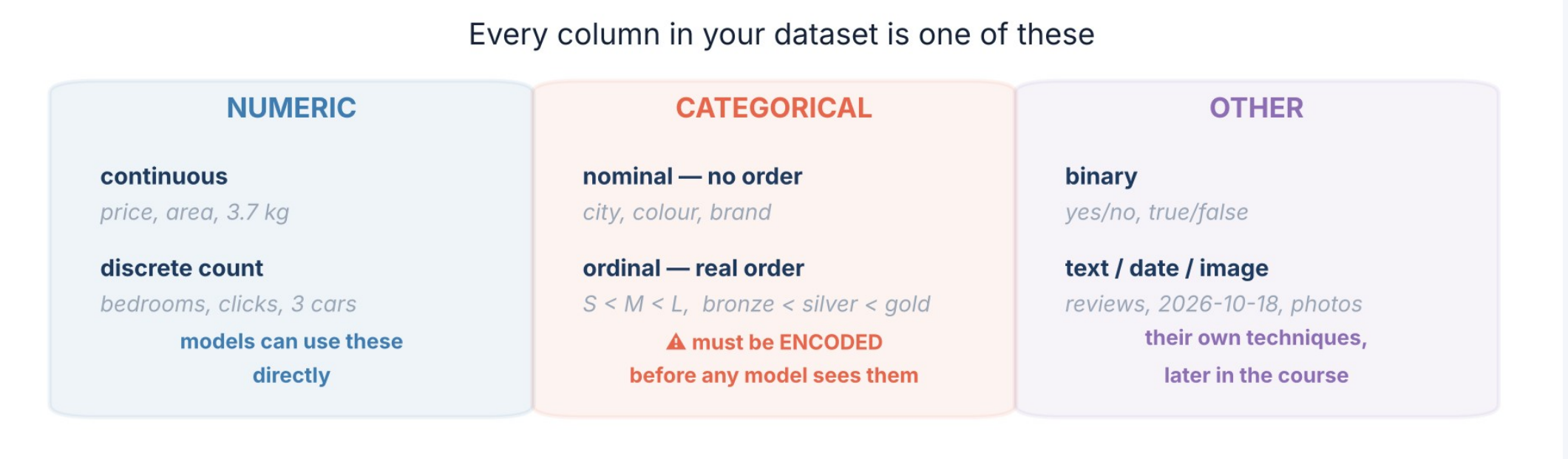
What a Feature Is

And why a model cannot read “Boston” · 10 minutes

- The types of column
- The problem with words

Slides by Srinivasa Sai Chava

Every column in your data is one of these



The distinction that matters today

NOMINAL means the categories have no order — Boston is not more or less than Miami. ORDINAL means they genuinely do — small really is less than large. Getting this one call wrong is the mistake in section 5, and it costs real accuracy.

Why you cannot just hand a model the word “Boston”

```
>>> from sklearn.linear_model import LinearRegression
>>> df[["area", "city"]].head(2)
    area    city
0   1450  Boston
1    890  Austin

>>> LinearRegression().fit(df[["area", "city"]], df.price)

ValueError: could not convert string to float: 'Boston'
```

Why it refuses

Every model you will meet does arithmetic on its inputs: it multiplies them by weights and adds them up.

There is no sensible value for “Boston × 0.4”.

So every categorical column must become numbers before training — and HOW you do that changes the answer.

This is not a software limitation you could code around. It is what the model IS: a weighted sum. Turning categories into numbers that the arithmetic can handle honestly is called ENCODING, and it is the rest of this session.

5

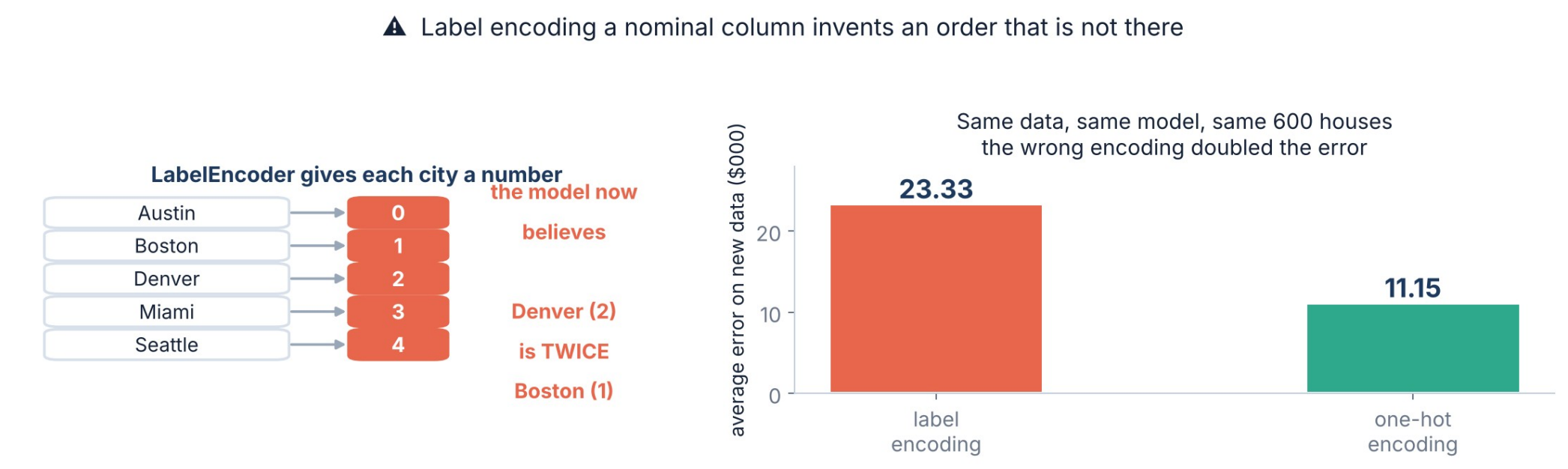
Encoding Categorical Data

Turning words into numbers, honestly · 20 minutes

- Label encoding and its trap
- One-hot
- Ordinal
- High cardinality
- Leakage

Slides by Srinivasa Sai Chava

⚠️ The obvious idea, and why it is wrong



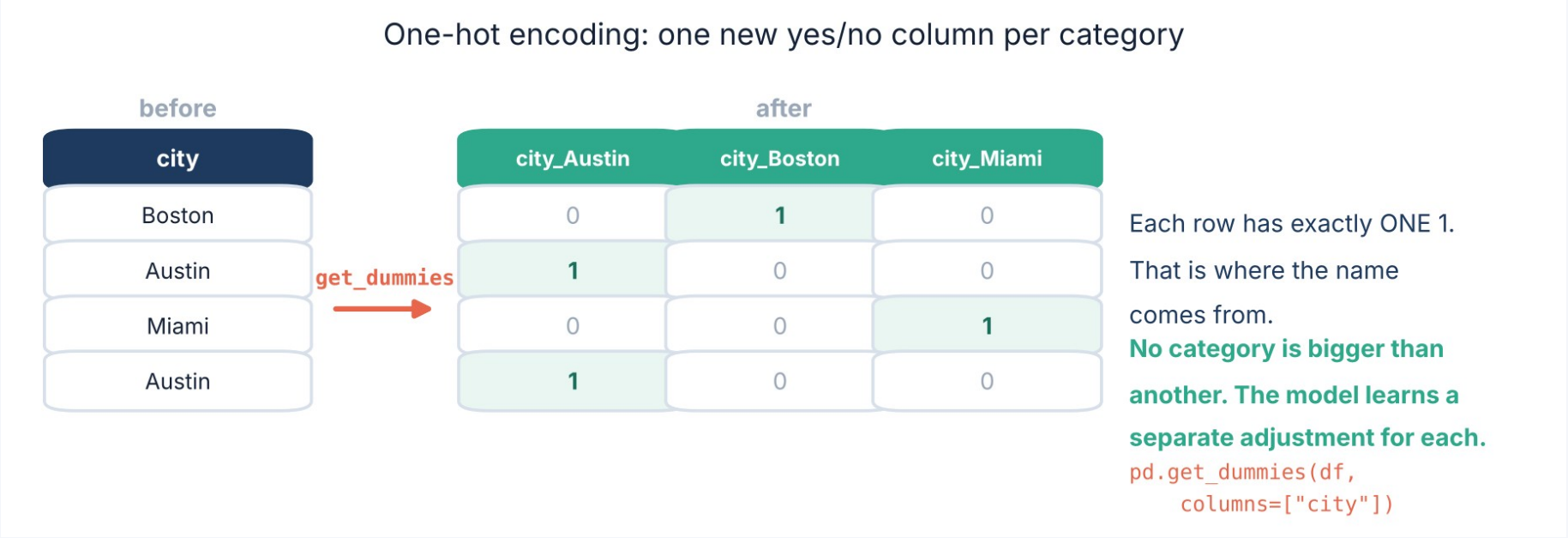
What the model now believes

Austin=0, Boston=1, Denver=2. A linear model multiplies that by a weight, so it is forced to believe Denver is twice Boston and Seattle is four times it. None of that is true.

Measured, on 600 houses

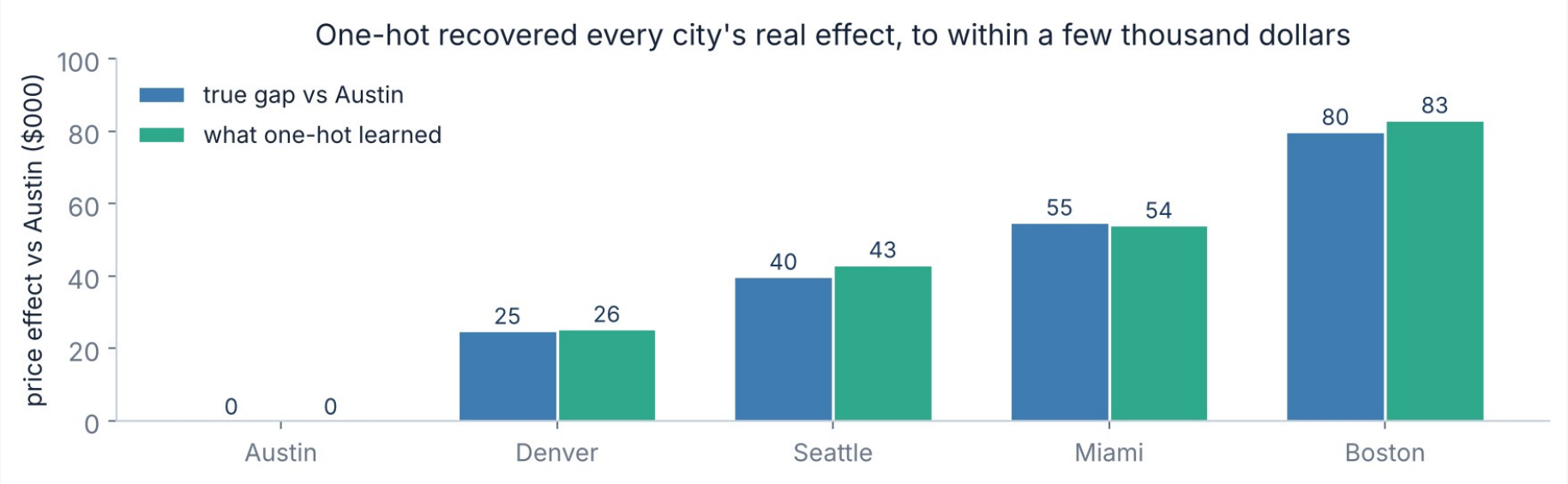
Same data, same model, only the encoding changed: average error on new houses went from $23,330 down to $11,150. The wrong encoding DOUBLED the error.

⭐ One-hot encoding — the right default for nominal columns



Each category gets its own yes/no column. Nothing is bigger than anything else, so no false order can be invented — and the model gets to learn a completely separate adjustment for each city, instead of being forced onto one straight line through them.

Proof that it worked: the model found the real answer



How we can check at all

This data was simulated, so we KNOW the true effect of each city — we built them in. Real data never lets you do this, which is exactly why simulated data is so useful for learning.

Every city within a few thousand dollars

Boston's true premium over Austin is $80k; the model learned $83k. Denver's is $25k; it learned $26k. It worked out five separate city effects from nothing but the one-hot columns.

The dummy variable trap — and what it really costs

The redundancy

If a house is not Austin, not Denver, not Miami and not Seattle, it must be Boston. The fifth column carries no information the other four did not already give you.

The five dummies add to exactly 1 in every row — which is the same thing the model's intercept already is.

Measured on our 600 houses

design matrix	cols	rank
intercept + all 5 dummies	7	6 ← deficient
intercept + 4 dummies	6	6 ← fine

Condition number with all five: 7.9×10^{18}. statsmodels warns outright.

The honest version of this rule — which most tutorials get wrong

It does NOT break your predictions. We checked: all five dummies and four dummies gave test error of $11,151 either way, identical to four decimal places, because scikit-learn's solver quietly picks one of the infinitely many equivalent answers.

What it breaks is INTERPRETATION — the coefficients are no longer unique, so reading them is meaningless — and classical OLS, which needs to invert a matrix that is now singular. Use drop_first=True whenever you plan to read the coefficients or use statsmodels. For a tree or a random forest it genuinely does not matter.

Ordinal encoding — when the order IS real

Some categories really are ranked

High School < Bachelor < Master < PhD

S < M < L < XL

bronze < silver < gold

never < sometimes < often < always

Here 0, 1, 2, 3 is not a lie — it is the truth, and it costs you ONE column instead of four.

⚠️ But it assumes EVEN spacing

0, 1, 2, 3 says the jump from Bachelor to Master is the same size as the jump from Master to PhD. Sometimes true, often not.

Both tested on 500 people, test error in $000:

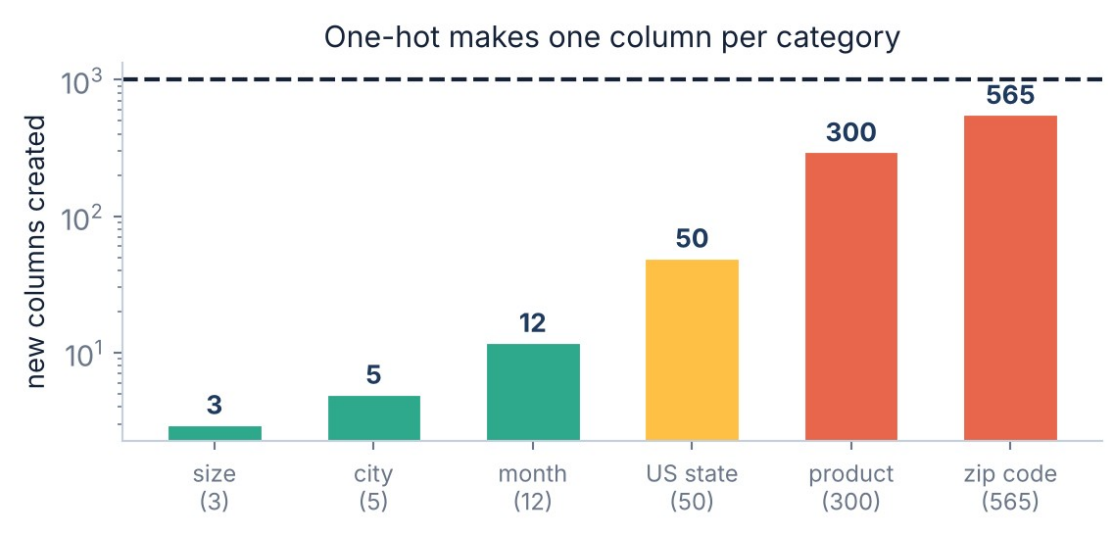
true gaps	ordinal	one-hot
even (0, 8, 16, 24)	4.98	4.97
uneven (0, 6, 9, 34)	6.04	4.64

The rule that follows from those two rows

When the spacing really is even, ordinal ties one-hot exactly (4.98 vs 4.97) and uses a quarter of the columns — use it. When the spacing is uneven, ordinal is 30% worse (6.04 vs 4.64), because it is forced to put the four levels on one straight line and the real jump to PhD is nothing like the others.

If you are unsure whether the gaps are even, one-hot is the safe choice: it never assumes anything about spacing.

⚠️ When one-hot is the wrong answer



1,000 rows, a zip-code column with 565 values

1,000 rows

565 columns

Almost every column is zero almost everywhere.

The model has nearly as many knobs as examples,

and it will memorise instead of learning.

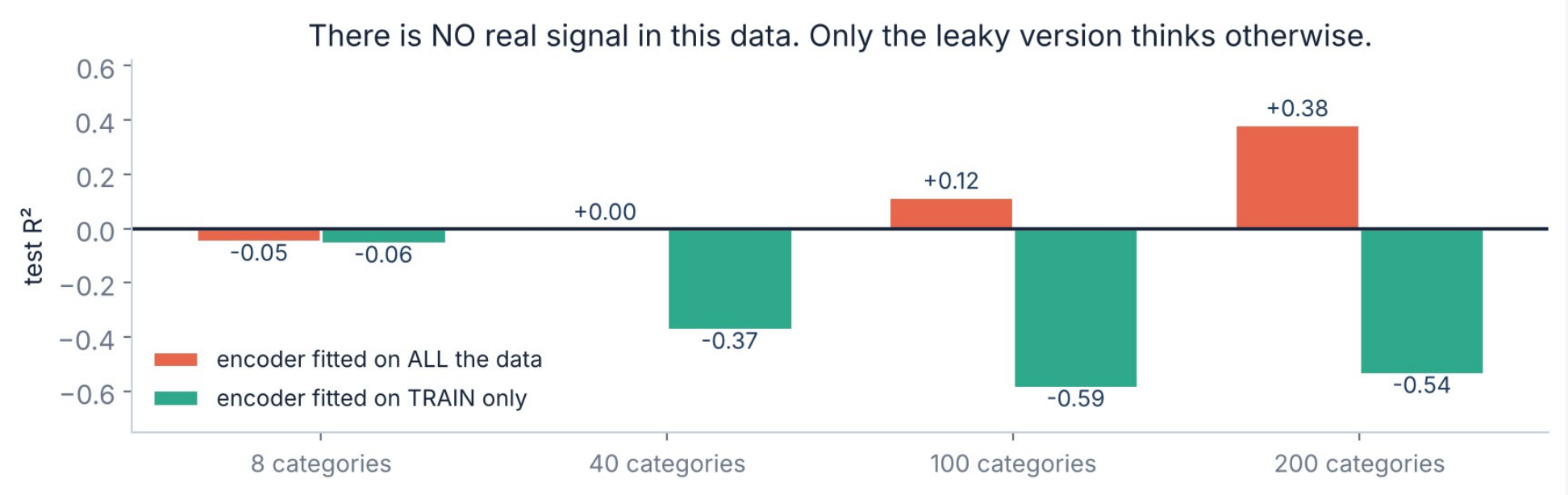
The blow-up

One-hot makes one column per category, so a zip-code column with 565 values turns 1,000 rows into a table with 565 extra columns — almost all zeros, and nearly as many knobs as there are examples to set them with. The model memorises.

What to do instead

Group the rare ones into “Other” and one-hot the rest · use FREQUENCY encoding (replace each category by how often it appears) · use TARGET encoding (replace it by the average label for that category) — with the warning on the next slide · or use a model that handles categories natively, such as CatBoost or LightGBM.

⚠️ Fit the encoder on TRAIN only — or it will lie to you



What the red bars are doing

The target encoder was fitted on all the rows, so each test row's own answer helped build its own feature. At 200 categories the model reports a test R^2 of +0.38 on data containing nothing at all.

The green bars are correct

Fitted on train only, the model reports a NEGATIVE test R^2 — correctly saying "this is worthless". The honest answer looks like failure, and the leaky one looks like success. That is the danger.

Which encoder, and when

Your column	Use	Code
Nominal, under ~15 categories	one-hot	`pd.get_dummies(df, columns=[c])`
Nominal, and you will read coefficients	one-hot, drop one	`pd.get_dummies(…, drop_first=True)`
Ordinal, gaps roughly even	ordinal — map by hand	`df[c].map({"S":0, "M":1, "L":2})`
Ordinal, gaps clearly uneven	one-hot (safer)	`pd.get_dummies(df, columns=[c])`
Nominal, hundreds of categories	group rare → Other, then one-hot	`value_counts() then .where()`
Still too many after grouping	frequency or target encoding	`df[c].map(df[c].value_counts())`
The LABEL of a classifier	label encoding is correct here	`LabelEncoder().fit_transform(y)`

Two rules that cover almost everything: when in doubt, ONE-HOT — it assumes nothing. And whatever you choose, FIT IT ON TRAIN ONLY. The last row is the exception worth remembering: LabelEncoder is the right tool for the thing you are predicting, and the wrong tool for the things you predict from.

RECAP

Debug corner — eight first-week mistakes

You did	What goes wrong	Fix
fed a string column to .fit()	ValueError: could not convert to float	encode it first
LabelEncoder on a nominal feature	error doubled: $11k → $23k	one-hot for nominal
one-hot on 565 zip codes	565 columns from 1,000 rows	group rare → Other
scored the model on its training data	R^2 = 1.000 and useless	always hold out a test set
fitted the encoder before splitting	fake test R^2 of +0.38 on noise	fit on train, apply to test
ordinal-encoded an uneven order	6.04 vs 4.64 error	one-hot when gaps are uneven
kept all k dummies, then read coefficients	coefficients are not unique	drop_first=True
no labels, reached for a classifier	nothing to learn from	it is an unsupervised problem

Rows 4 and 5 are the same mistake wearing different clothes: letting the model see, in any form at all, data you intend to grade it on.

Putting it together — prepare any dataset for a model

```python
import pandas as pd
from sklearn.model_selection import train_test_split

def prepare(df, label, ordinal_maps=None, max_cats=15):
    """Split first, then encode. Returns X_train, X_test, y_train, y_test."""
    ordinal_maps = ordinal_maps or {}
    y = df[label]
    X = df.drop(columns=[label])

    #  1. SPLIT FIRST - nothing may learn from the test rows
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.25, random_state=42)

    #  2. ordinal columns: a map YOU wrote, so order is intentional
    for col, mapping in ordinal_maps.items():
        X_tr[col] = X_tr[col].map(mapping)
        X_te[col] = X_te[col].map(mapping)

    #  3. group rare categories, using TRAIN counts only
    cats = X_tr.select_dtypes(include="object").columns
    for col in cats:
        keep = X_tr[col].value_counts().head(max_cats).index
        X_tr[col] = X_tr[col].where(X_tr[col].isin(keep), "Other")
        X_te[col] = X_te[col].where(X_te[col].isin(keep), "Other")

    #  4. one-hot, then align so both frames have the same columns
    X_tr = pd.get_dummies(X_tr, columns=list(cats)).astype(float)
    X_te = pd.get_dummies(X_te, columns=list(cats)).astype(float)
    X_te = X_te.reindex(columns=X_tr.columns, fill_value=0.0)
    return X_tr, X_te, y_tr, y_te
```

Split is the FIRST step

everything after it is fitted on train

Ordinal maps are hand-written

an order you chose, not one invented

Rare categories use TRAIN counts

test rows must not vote

Columns are aligned at the end

a city only in test would break it

astype(float)

booleans trip up some models

No label encoding anywhere

except on y, which is not here

In real projects a scikit-learn Pipeline does all of this for you, and refuses to let you leak. We build it by hand first so you know what it is doing.

OVER TO YOU

Practice session — 30 minutes

A **Predict the output** — 8 questions

Types, encodings, shapes, splits

B **Spot & fix the bug** — 4 questions

Label on nominal, leak, scoring on train

C **Write the code** — 5 tasks

Encode, split, prepare, compare

D **Challenge** — 2 tasks

Classify 8 real problems, then fix a pipeline

Your notebook

`Day31_Practice_Questions.ipynb`

- Split before you encode
- Nominal → one-hot
- Never score on training data
- Solutions shared after class

No model fitting is required today beyond what is given to you. Today is about getting data into a shape a model can actually use — which, as the pipeline slide showed, is most of the real work.

TODAY IN TWELVE LINES

Recap

1. ⭐ Traditional code: rules + data → answers. ML: data + answers → rules.
2. Sample = a row. Feature = an input column. Label = the answer column.
3. “Learning” = try numbers, measure the error, keep the best. No magic.
4. SUPERVISED: the answers come with the data. ~90% of industry work.
5. Regression = the label is a number. Classification = it is a category.
6. UNSUPERVISED: no answers — cluster the rows, or shrink the columns.
7. REINFORCEMENT: no answers, but the world pays you a reward.
8. ⚠️ A tree scored R^2 = 1.000 on its training data and −0.628 on new data.
9. ⚠️ Models do arithmetic, so every word column must be ENCODED first.
10. ⚠️ Label-encoding a nominal column doubled the error: $11k → $23k.
11. One-hot: one yes/no column each. No false order. The safe default.
12. ⚠️ Split FIRST, then fit encoders on train only — or your score is a lie.

Homework

Take your Day 21 dataset: list every column and label it numeric / nominal / ordinal · say what a supervised problem on it would predict, and whether that is regression or classification · run prepare() on it and report the shape before and after

Next class — Topics 3.3 + 3.4

Feature scaling, and feature engineering. Why a column measured in thousands drowns out one measured in units, what to do about it, and how to build features that are not in the data you were given.